Gilmore Girls Monopoly Game!!!

Program Overview

This program simulates a monopoly style game using a circular singly linked list. The board contains ten properties based on locations from the show the Gilmore Girl's, the players are set to "move around" the board, purchase available properties, and receive a final summary of their money and properties.

Code Explanation

The Property structure represents one property on the board. Each property stores its name, cost, owner, and a pointer to the next property.

The PropertyLinkedList class manages the properties. It includes:

- add(): Adds a property to the list
- search(): Finds a property by name
- remove(): Deletes a property
- print(): Displays all properties
- makeCircular(): Connects the last property back to the first
- breakCircularLink(): Disconnects the list before it is destroyed

The list uses both a head and a tail pointer. The head points to the first property, while the tail points to the last property. The size variable stores the number of properties in the list.

The circular board class creates the game board by adding ten properties to the linked list. It then makes the list circular, allowing players to move from the last property back to the first property.

The player structure stores each player's name, money, and current location. Each player begins with $1500 and starts at the first property on the board.

The testLinkedList() function tests the main linked list operations. It adds properties, searches for a property, removes a property, and prints the list after each operation.

The playGame() function runs the game simulation. It uses predetermined dice rolls for ten turns. Each turn, the current player moves through the circular linked list. If the property is unowned and the player has enough money, the player purchases it. At the end of the game, the program displays the remaining money and owned properties for each of the players.

The main() function runs the linked list tests, creating a circular board, printing the “board”, and commences the game.

Time Complexity

Operation	**Time Complexity**
Adding a property	O(1)
Searching for a property	O(n)
Removing a property	O(n)
Moving over a space	O(1)
Moving ‘r’ spaces	O(r)
Traversing the list	O(n)

Adding

Adding a property takes O(1) time, because the program has a tail pointer, so the new property can be added directly to the end of the list without searching through the other nodes.

Searching

Searching takes O(n) time in the worst case, because the program starts at the head and checks each property one at a time until it finds the requested name. If the property is at the end or does not exist, every property must be checked.

Removing

Removing a property takes O(n) time in the worst case, because the program must first search for the property and keep track of the previous node. Once the property is found, changing the links and deleting the node takes O(1) time. The search makes the overall operation O(n).

Moving

Moving one space takes O(1) time because the player’s position is changed using the current property’s next pointer. Moving r spaces takes O(r) time because the program follows the pointer once for each space.

Because the board is circular, the player can move from the last property back to the first property without searching for the beginning of the board.

Traversing

Traversing the list takes O(n) time because the program visits each property once. This occurs when the program prints the board, prints the linked list, or checks which properties belong to each player.

Summary

Therefore the program demonstrates how a circular singly linked list can be used to represent a board game. Adding and moving are efficient because the program uses pointers to access positions directly. Searching, removing, and traversing may require visiting multiple nodes, so their running time depends on the number of properties in the list.